

Sionna RT 技术问答

本文档逐一回答师兄提出的技术问题。所有结论均基于 Sionna RT v2.0.1 源码和教程分析。

Q1: 生成的信道图和 pathloss 图是不是重叠的，还是可以每个基站各自拆开？

答：可以拆开，每个基站信道图是独立存储的。

`RadioMapSolver` 输出的 `PlanarRadioMap.path_gain` 张量 shape 为 `[num_tx, cells_y, cells_x]`，每一个发射机（基站）有自己独立的二维路径增益矩阵。

代码验证

```
# 放置3个基站
scene.add(Transmitter(name="bs_0", position=[10, 0, 10], power_dbm=44))
scene.add(Transmitter(name="bs_1", position=[-10, 0, 10], power_dbm=44))
scene.add(Transmitter(name="bs_2", position=[0, 10, 10], power_dbm=44))

# 计算无线电地图
rm = RadioMapSolver()(scene, ...)

# 各基站独立数据
bs0_gain = rm.path_gain[0] # shape [cells_y, cells_x] - BS 0 的图
bs1_gain = rm.path_gain[1] # BS 1 的图
bs2_gain = rm.path_gain[2] # BS 2 的图

# 可视化：按发射机索引筛选
scene.render(camera=cam, radio_map=rm, rm_tx=0) # 仅显示 BS 0
scene.render(camera=cam, radio_map=rm, rm_tx="bs_1") # 仅显示 BS 1
scene.render(camera=cam, radio_map=rm, rm_tx=None) # 全部叠加（取每个格点的最大值）
```

三种无线电地图指标

指标	rm_metric	含义	各基站关系
路径增益	"path_gain"	各路径 $ a ^2$ 在每个格点的空间平均	独立存储, <code>shape[0] = num_tx</code>
接收信号强度	"rss"	<code>path_gain × transmit_power → dBm</code>	独立存储
信干噪比	"sinr"	考虑多 BS 互干扰和热噪声	天然交叉 (信号来自目标 BS, 干扰来自其他 BS)

结论

信道图和 pathloss 图在数据层面天然可拆分——每个基站有自己的二维数据矩阵。视觉化时通过 `rm_tx` 参数可任意选择查看单个基站或合成图。

Q2: 各个基站设置的天线数目是否可以相同或不同?

答: Sionna RT v2.0.1 原生不支持不同基站配置不同天线阵列。

架构原因

`scene.tx_array` 是一个全局单例属性, 所有 `Transmitter` 共用同一个 `AntennaArray` 配置。关键代码路径:

- `scene.sources()` 方法调用 `scene.tx_array.antenna_pattern.patterns` 生成所有 TX 的源天线方向图 —— 所有发射机使用同一组方向图
- `RadioMapSolver` 中使用 `scene.tx_array.num_ant` 统一处理所有发射机

变通方案

方案 A (推荐): 分次运行

```
# 第一次运行: BS1 使用 1x1 全向天线
scene.tx_array = PlanarArray(num_rows=1, num_cols=1, pattern="iso", polarization="V")
scene.add(Transmitter(name="bs_0", ...))
# 只留 bs_0, 计算并保存 rm_0

# 第二次运行: BS2 使用 2x16 定向天线
```

```
scene.clear()
scene.tx_array = PlanarArray(num_rows=2, num_cols=16, pattern="tr38901", polarization="V")
scene.add(Transmitter(name="bs_1", ...))
# 计算并保存 rm_1
```

方案 B：修改 Sionna RT 源码 在 `scene.py` 中将 `tx_array` 从全局属性改为每个 `Transmitter` 可独立覆盖的属性。需要修改 `sources()` 方法和 `RadioMapSolver` 的天线查询逻辑。工作量大，不推荐。

方案 C：使用预编码向量模拟 如果两个基站的天线数目差异不大，可以在 `RadioMapSolver` 中通过 `precoding_vec` 参数对不同基站使用不同预编码向量（但底层天线数目仍相同）。

结论

这是 Sionna RT 已知的架构限制。对研究场景，分次运行（方案 A）是最实用的变通手段。

Q3: 生成的信道矩阵在什么地方，通过什么方式修改成了信道图？

答：Sionna RT 中存在三层数据，不存在直接的"信道矩阵 → 信道图"转换。

三层数据结构

Layer 1: 路径级数据 (Paths 对象)

`PathSolver.__call__()` 返回，包含每条传播路径的物理参数：

属性	含义	Shape
<code>paths.a</code>	复通带路径系数（琼斯矩阵）	<code>[num_rx, num_tx, num_paths]</code>
<code>paths.tau</code>	路径时延 [s]	<code>[num_rx, num_tx, num_paths]</code>
<code>paths.doppler</code>	多普勒频移 [Hz]	<code>[num_rx, num_tx, num_paths]</code>
<code>paths.theta_t/phi_t</code>	离开角 (天顶/方位) [rad]	<code>[num_rx, num_tx, num_paths]</code>
<code>paths.theta_r/phi_r</code>	到达角 (天顶/方位) [rad]	<code>[num_rx, num_tx, num_paths]</code>

"信道矩阵"如果存在的话，最接近 `paths.a` —— 它是每条路径的复增益（包含幅度和相位）。

Layer 2: CIR（信道冲激响应）

```
a, tau = paths.cir(sampling_frequency=122.88e6, normalize_delays=False)
```

将路径数据采样为离散时间冲激响应： - `a`: [num_rx, num_time_steps, num_tx, num_tx_ants, num_paths, num_rx_ants] - `tau`: [num_rx, num_time_steps, num_tx, num_tx_ants, num_paths]

基带转换公式（在 `paths.cir()` 内部完成）：
$$a_i^b = a_i \cdot e^{-j 2\pi f \tau_i} \cdot e^{j 2\pi f_{\Delta,i} t}$$

其中 a_i 是通带路径系数， f 是载频， $f_{\Delta,i}$ 是多普勒频移。

CIR 是时域信道矩阵：对固定时刻 t ，`a[:, t, :, :, :]` 就是这组天线对之间所有路径的复增益矩阵。

Layer 3: 信道图（RadioMap 对象）

`RadioMapSolver.__call__()` 返回，是空间采样的标量路径增益图：

```
path_gain: [num_tx, cells_y, cells_x]
```

每个格点 (i,j) 的值是该位置对所有到达路径的 $|a|^2$ 的空间加权平均。

关键区分

	Paths / CIR	RadioMap
粒度	每条路径独立	空间格点综合
数据类型	复系数（幅度+相位）	标量（功率）
空间维度	针对特定发射机-接收机对	覆盖整个空间平面
生成方式	PathSolver（射线追踪）	RadioMapSolver（蒙特卡洛空间采样）

不存在从 Paths/CIR "转换"为 RadioMap 的过程 —— RadioMap 是通过独立的蒙特卡洛采样直接计算的，每个格点对应一个虚拟接收机位置，对该位置的所有到达射线做功率累加。

如果想要完整的信道矩阵（非标量图）

需要自己写代码将 CIR 扩展为空间矩阵：

```
# 对每个虚拟接收机位置，运行一次 PathSolver，得到 paths.a
# 将所有位置的 paths.a 组合为 [num_rx_positions, num_tx, num_paths] 的信道张量
```

Q4: 有没有带上导频信息的信道?

答: **Sionna RT** 原生不生成导频结构。

`paths.cir()` 输出的 CIR 是裸物理层信道冲激响应，不包含任何协议层信息（导频、同步序列、帧结构等）。

项目现有能力: CIR → CFR + 导频叠加

项目 `tools/CIR_to_CFR/` 目录已实现完整的 OFDM 导频处理链路:

1. **CIR → CFR 转换:**
 2. 将时域 CIR 通过 FFT 转换到频域 CFR
 3. 支持 OFDM 子载波间隔配置 (如 120 kHz)
 4. 支持子载波数目配置 (如 1024 子载波)
5. **导频掩码生成:**
 6. 生成 `[T, K]` 布尔数组 (T: OFDM 符号数, K: 子载波数)
 7. True 位置插入导频符号, False 位置为数据符号
8. **LS 信道估计:**
 9. 基于导频位置的最小二乘信道估计
10. 支持插值恢复全频带信道

```
from tools.CIR_to_CFR.dataset_CIR_to_CFR import CIRDataset, cirs_to_cfr

# CIR → CFR 转换
cfr = cirs_to_cfr(a_static, tau_static, K=1024, scs=120e3)

# 叠加导频结构
pilot_mask = generate_pilot_mask(T=14, K=1024, pilot_pattern="comb")
cfr_with_pilots = cfr * pilot_mask # 数据位置置零, 仅保留导频
```

结论

Sionna RT 提供裸物理层信道；导频等协议层信息需要在后处理阶段叠加。项目已有成熟的 CIR→CFR 工具链支持。

Q5: 主要采用 2 个频段 3.5GHz 和 28GHz —— Sionna RT 如何支持？

答：两个频段均可通过 `scene.frequency` 直接设置，Sionna RT 自动处理频率相关的物理效应。

设置方式

```
# 3.5 GHz (Sub-6)
scene.frequency = 3.5e9

# 28 GHz (mmWave)
scene.frequency = 28e9
```

频率切换时的自动更新

设置 `scene.frequency` 会触发：1. `scene.wavelength` = c / f 更新 2. `scene.wavenumber` = $2\pi / \lambda$ 更新 3. `scene.thermal_noise_power` = $k \cdot T \cdot B$ 重新计算 4. 所有无线电材料的频率属性更新：`RadioMaterial.frequency_update()` 重新计算该频率下的复介电常数、反射系数、透射系数

两频段关键差异

特性	3.5 GHz	28 GHz
波长	85.7 mm	10.7 mm
自由空间路径损耗 (@100m)	~83 dB	~101 dB
穿透能力	较好（可穿墙）	差（易被遮挡）
衍射能力	较强	弱
漫反射	较弱	较强（短波长表面粗糙度影响大）
微多普勒调制指数 β (@R=0.15m)	~22 rad	~176 rad
峰值微多普勒频偏 (@f_rot=100Hz)	~±2.2 kHz	~±17.6 kHz

建议的射线追踪参数差异

参数	3.5 GHz	28 GHz
<code>max_depth</code>	5~10	3~5
<code>diffraction</code>	True（绕射重要）	False
<code>diffuse_reflection</code>	False（可选）	True（漫反射显著）
<code>cell_size</code> (RadioMap)	[2.0, 2.0] m	[0.5, 0.5] m

结论

Sionna RT 对双频段支持完善，仅需修改一行 `scene.frequency`。注意两频段的传播特性差异很大，射线追踪参数需分别调优。

Q6: Camera preview 是什么？

答： `scene.preview()` 是 Sionna RT 提供的交互式 3D 场景查看器，可在 Jupyter Notebook 中实时查看场景、路径和无线电地图。

功能

- 1. 交互式 3D 导航：鼠标左键旋转、滚轮缩放、右键平移
- 2. 场景可视化：所有场景物体（建筑、地面等）和无线电设备（绿色 RX、蓝色 TX）
- 3. 路径覆盖：通过 `paths` 参数叠显射线追踪计算出的传播路径
- 4. 无线电地图热力图覆盖：通过 `radio_map` 参数在场景上叠加路径增益/RSS/SINR 热力图
- 5. 按基站筛选： `rm_tx` 参数切换显示特定基站的地图
- 6. 裁剪面： `clip_at` 参数剖切显示建筑内部
- 7. 坐标拾取：Alt+点击获取鼠标位置坐标

使用方式

```
import sionna
from sionna.rt import load_scene, Transmitter, Receiver, PlanarArray, Camera

scene = load_scene(sionna.rt.scene.etoile)
scene.frequency = 28e9
scene.tx_array = PlanarArray(num_rows=1, num_cols=1, pattern="iso", polarization="V")
```

```
# 添加设备
scene.add(Transmitter(name="bs", position=[50, 0, 30], power_dbm=44))
scene.add(Receiver(name="ue", position=[0, 0, 1.5]))

# 交互式预览（仅在 Jupyter 中有效）
scene.preview()

# 创建 Camera 对象定义视点
camera = Camera(position=[100, 50, 60])
camera.look_at([0, 0, 10])

# 静态渲染（可保存）
scene.render(camera=camera, resolution=[1920, 1080], num_samples=256)

# 渲染到文件
scene.render_to_file(camera=camera, filename="scene.png")
```

与传统摄像头的类比

概念	Camera preview	物理摄像头
视点	<code>Camera.position</code>	摄像头位置
朝向	<code>camera.look_at(target)</code>	镜头方向
视场角	<code>fov</code> 参数 (默认 45°)	镜头焦距
分辨率	<code>resolution</code> 参数	传感器像素
画质	<code>num_samples</code> (光线采样数)	曝光时间/ISO

Camera 类

```
from sionna.rt import Camera

camera = Camera(position=[x, y, z], orientation=[alpha, beta, gamma])
camera.look_at(target) # 让相机指向目标点
```


结论

`scene.preview()` 是开发和调试的必备工具——可以直观看到场景结构、设备位置、路径走线、以及无线电地图覆盖。但它仅在 Jupyter 环境中可用。对于命令行脚本，使用 `scene.render()` 或 `scene.render_to_file()` 生成静态图片。

Q7: 动态的时候如果有两个物体都在移动，可不可以成功建模出来？

答：可以。**Sionna RT** 支持多物体同时移动的建模，提供两种方式。

方式 1：逐步位移 + 重追踪（精确，适合大范围移动）

每步更新所有运动物体的位置，重新执行射线追踪：

```
# 两个物体同时移动
drone_velocity = [1.0, 0.5, 0.0]
car_velocity = [0.0, 15.0, 0.0]
dt = 0.1

for step in range(num_steps):
    # 更新无人机位置
    scene.get("drone").position += drone_velocity * dt

    # 更新汽车位置
    scene.get("car").position += car_velocity * dt

    # 重新追踪所有路径
    paths = solver(scene, max_depth=5, ...)

    # 这次追踪的 paths 自动反映两个物体移动后的新场景
    # paths.doppler 也会正确累加两个物体的速度贡献
```

原理：每个场景物体有 `velocity` 属性，在 PathSolver 内部计算每条路径上每个散射点的速度贡献：
$$\Delta = \frac{1}{\lambda} \left(\mathbf{k}_{rx} \cdot \mathbf{v}_{rx} - \mathbf{k}_{tx} \cdot \mathbf{v}_{tx} + \sum_i \mathbf{k}_i \cdot \mathbf{v}_i \right)$$

其中 $\mathbf{k}_i$ 是第 i 个散射点的出射方向， $\mathbf{v}_i$ 是该散射点所在物体的速度。

方式 2：设置 velocity + 利用 Doppler 重建（快速，适合小范围移动）

一次性 RT 求解，通过 `paths.doppler` 和 `paths.cir(num_time_steps=N)` 重建时间演化：

```
# 设置两个物体的速度
scene.get("drone").velocity = [1.0, 0.5, 0.0]
scene.get("car").velocity = [0.0, 15.0, 0.0]

# 一次性射线追踪（static scene geometry）
paths = solver(scene, ...)

# 用 Doppler 信息生成 time-evolving CIR（物体不实际移动）
a_timed, tau_timed = paths.cir(sampling_frequency=122.88e6, num_time_steps=100)

# a_timed shape: [num_rx, 100, num_tx, num_tx_ants, num_paths, num_rx_ants]
```

限制：方式 2 假设物体位移在几个波长内——路径时延和角度不变化，仅相位因多普勒频移而变化。对于大范围移动（如 UAV 飞过整个场景），必须用方式 1。

无人机微多普勒的特殊情况

无人机旋翼的微多普勒**不能**简单地通过设置 `velocity` 来建模——叶片运动是旋转而非平移，产生的相位调制是非线性的（正弦调制 $\exp(j\beta\sin(\omega t))$ ），不是恒定多普勒频移。

推荐方案：方式 1 + 项目现有的 `MicroDopplerModulator`：

1. 用方式 1 逐步更新无人机机身位置（平移运动）
2. 叶片旋转的微多普勒在 CIR 后处理阶段通过 `MicroDopplerModulator.modulate_cir()` 叠加
3. 这样机身大范围移动 + 叶片微多普勒都能正确建模

结论

两物体同时移动是 Sionna RT 的标准能力。选择精确方式还是快速方式取决于运动幅度。无人机的微多普勒需要额外叠加（项目 Step 5 已验证的 CIR 调制方法）。